

NAME: Gabriel Gabriel Malisa
REGISTRATION NO: 230222465417

Question 1

"Rank the following complexities in terms of their rate of growth starting with slowest to fastest: $O(n)$, $O(\log n)$, $O(n^3)$, $O(n \log n)$, $O(1)$."

- $O(1)$ – constant, never changes regardless of n
- $O(\log n)$ – grows very slowly (e.g., $\log_2$ of 1,000,000 is only ~ 20)
- $O(n)$ – grows linearly, directly proportional to n
- $O(n \log n)$ – slightly worse than linear, n multiplied by a slowly growing factor
- $O(n^3)$ – grows very fast, e.g., $n=100$ gives 1,000,000 operations

Answer:

$O(1) < O(\log n) < O(n) < O(n \log n) < O(n^3)$

Question 2: Find the Big-O complexity

a) $4n^2 + 20n + 9$

In Big-O we focus on the **dominant term**, the one that grows fastest as $n \rightarrow \infty$. Here we have three terms: $4n^2$, $20n$, and 9 . As n gets large, n^2 dominates n which dominates the constant. We also drop the coefficient 4.

$\therefore O(n^2)$

b) $25n^3 + 15$

Two terms: $25n^3$ and constant 15. The cubic term dominates completely for large n , and we drop the coefficient 25.

$\therefore O(n^3)$



c) $n^3 + 5\log n + 6$

Three terms: n^3 , $5\log n$, and 6. Comparing growth rates, n^3 grows far faster than $\log n$ for large n , so n^3 dominates. Drop the lower terms.

'n $O(n^3)$

d) $10^5 + 10^5n + 10^5n^2$

Three terms, all with the same coefficient 10^5 (which we ignore). What matters is the powers of n : constant, n , and n^2 . The highest power n^2 dominates.

'n $O(n^2)$

e) $72n(n + 3n^2)$

First, expand the brackets:

$$72n \times n + 72n \times 3n^2 = 72n^2 + 216n^3$$

Now we have two terms. n^3 dominates n^2 , and we drop all coefficients.

'n $O(n^3)$

f) $2^8 + 32^4n - 5$

$2^8 = 256$ and 5 are both pure constants. 32^4 is also just a large constant coefficient on n . So we really have:

constant + constant $\times n$ - constant

The only term involving n is linear, everything else is constant. Drop the coefficient.

'n $O(n)$



Question 3

"Design an algorithm that runs in worst-case $O(n)$ time to add all the numbers from 1 to a given number n (including n)."

Working: We need $O(n)$ meaning the algorithm should do roughly n steps. The simplest approach is a loop that runs from 1 to n , adding each number to a running total. Each addition is $O(1)$, and we do it exactly n times 'n $O(1) \times n = O(n)$.

```
sum := 0
for i := 1 to n do
  sum := sum + i
endfor
return sum
```

'n $O(n)$

Question 4: Worst-case running time

a) Complexity in terms of *number*.

```
x = 4
for i := 0 to number do
  x := x + 1
endfor
```

- $x = 4$ is a single assignment 'n $O(1)$
- The for loop runs from 0 to number, so it executes number times
- The body $x := x+1$ is a single operation 'n $O(1)$ per iteration
- Total: $O(1) + \text{number} \times O(1) = O(\text{number})$

'n $O(\text{number})$

b) Complexity in terms of *cycles*:

```
choice := get_choice()    //  $O(1)$  given
if choice=1 then
  print "Ready"           //  $O(1)$ 
elseif choice=2 then
  for i:=0 to cycles do
```



```

        print i          // O(cycles)
    endfor
endif

```

. For Big-O we always consider the **worst case** — the path through the code that takes the longest. The if-branch does $O(1)$ work, but the elseif-branch runs a loop *cycles* times. The worst case is when *choice=2* is selected, triggering the loop.

'n $O(\text{cycles})$

c) Complexity of *calculate* (passing *iter* to *increase*):

```

increase(val, iter):
    for i:=1 to iter do    // loops 'iter' times
        val := val + 5
    endfor
    return val            // 'n  $O(\text{iter})$ 

```

```

calculate(val, iter):
    for i:=1 to iter do    // loops 'iter' times
        val := val + increase(val, iter) // calls increase 'n  $O(\text{iter})$ 
    endfor
    return val

```

- First find *increase*: it loops *iter* times doing $O(1)$ work 'n $O(\text{iter})$
- Then analyse *calculate*: its loop runs *iter* times, and on **each** iteration it calls *increase* which costs $O(\text{iter})$
- Total = $\text{iter} \times O(\text{iter}) = O(\text{iter}^2)$

'n $O(\text{iter}^2)$



d) Complexity of *calculate* (passing *i* to *increase*):

```
calculate(val, iter):  
  for i:=1 to iter do  
    val := val + increase(val, i) // now passes i, not iter  
  endfor  
  return val
```

• *increase* is the same as before, but now it receives *i* instead of *iter*. So on each iteration the cost of *increase* changes:

- *i*=1: *increase* costs $O(1)$
- *i*=2: *increase* costs $O(2)$
- *i*=3: *increase* costs $O(3)$
- ...
- *i*=*iter*: *increase* costs $O(\text{iter})$

Total work = $O(1 + 2 + 3 + \dots + \text{iter}) = O(\text{iter}(\text{iter}+1)/2) = O(\text{iter}^2)$

The result is the same order as (c), though in practice this version does roughly **half the work** since the inner loop grows gradually rather than always running the full *iter* iterations.

'n $O(\text{iter}^2)$

